

Data Structures and Algorithms

Array Stack & Queue



Lecture 6

Software Engineering Dep.

Second Stage

Dr.Saman M. Omer

2025-2026

Outline

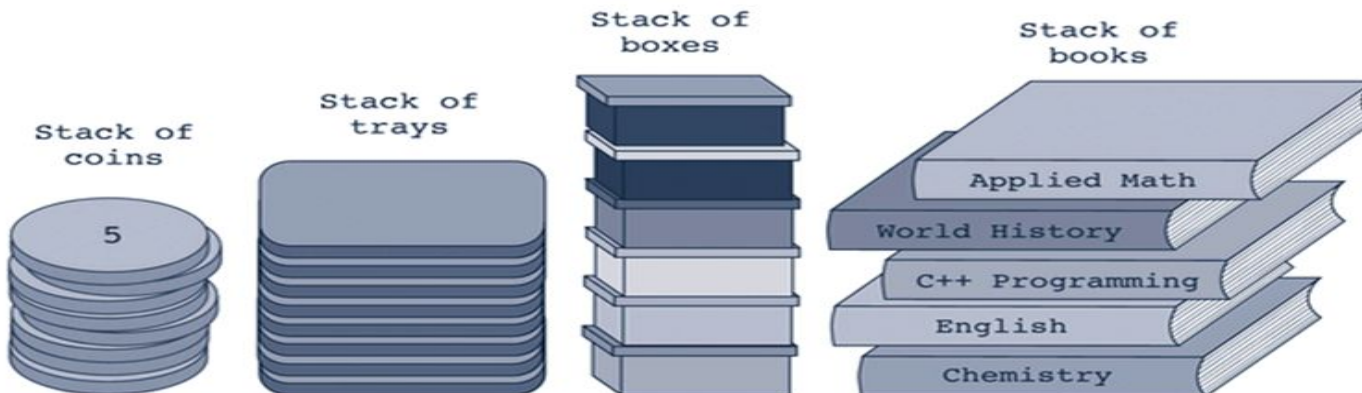
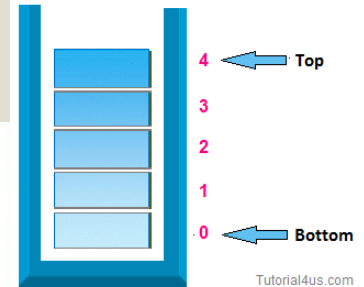
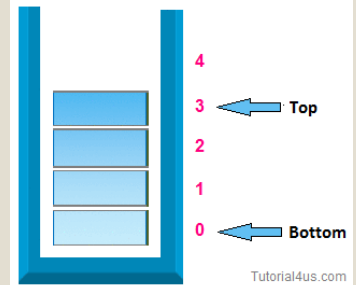
2

- Stack
- Stack Applications and Operations
- Array-Based Stack Implementation
- A Drawback and Analyzing the Array-Based Stack Implementation
- Queue
- Queues Applications and Operations
- Array based queue implementation
- Using an Array Circularly
- Analyzing the Efficiency of an Array-Based Queue
- Implementing a Queue with a Singly Linked List

Stack

3

- ❑ A **stack** is a collection of objects that are inserted and removed according to the **last-in, first-out (LIFO)** principle.
- ❑ A stack is a Last In First Out (**LIFO**), the last item added to the stack will be the first one to be removed.
- ❑ Only access or remove the most recently inserted object that remains at the “top” of the stack.
- ❑ The fundamental operations involve the “**pushing**” and “**popping**”.



Stack Applications



- ❑ Most microprocessors use a stack-based architecture. When a method is called, its return address and arguments are pushed onto a stack, and when it returns, they're popped off.
- ❑ **Example**: Internet Web browsers store the addresses of recently visited sites on a stack. Each time a user visits a new site, that site's address is “pushed” onto the stack of addresses. The browser then allows the user to “pop” back to previously visited sites using the “back” button.



- ❑ **Example** :Text editors usually provide an “*undo*” mechanism that cancels recent editing operations and reverts to former states of a document. This undo operation can be accomplished by keeping text changes in a stack.



Operations on Stack

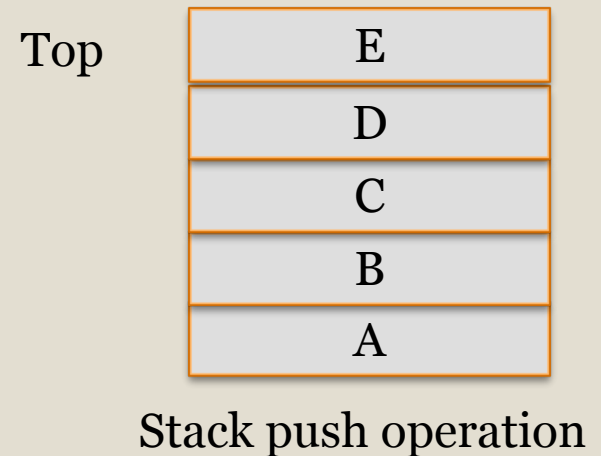
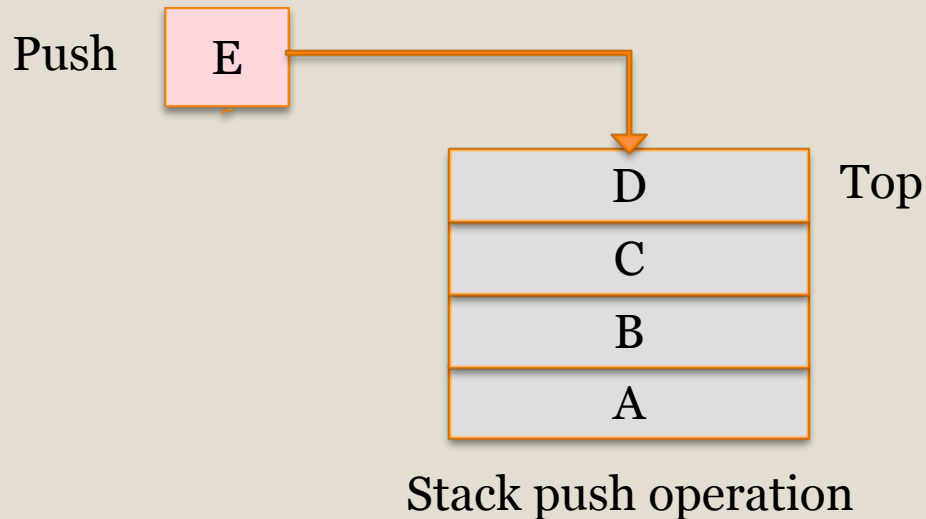


- ❑ Formally, a stack is an abstract data type (ADT) that supports the following two update operation methods:
 - **push(e):** Adds element e to the top of the stack.
 - **pop():** Removes and returns the top element from the stack (or null if the stack is empty).
- ❑ Additionally, a stack supports the following accessor methods for convenience:
 - **top():** Returns the top element of the stack, without removing it (or null if the stack is empty).
 - **size():** Returns the number of elements in the stack.
 - **isEmpty():** Returns a Boolean indicating whether the stack is empty.

Push Operation



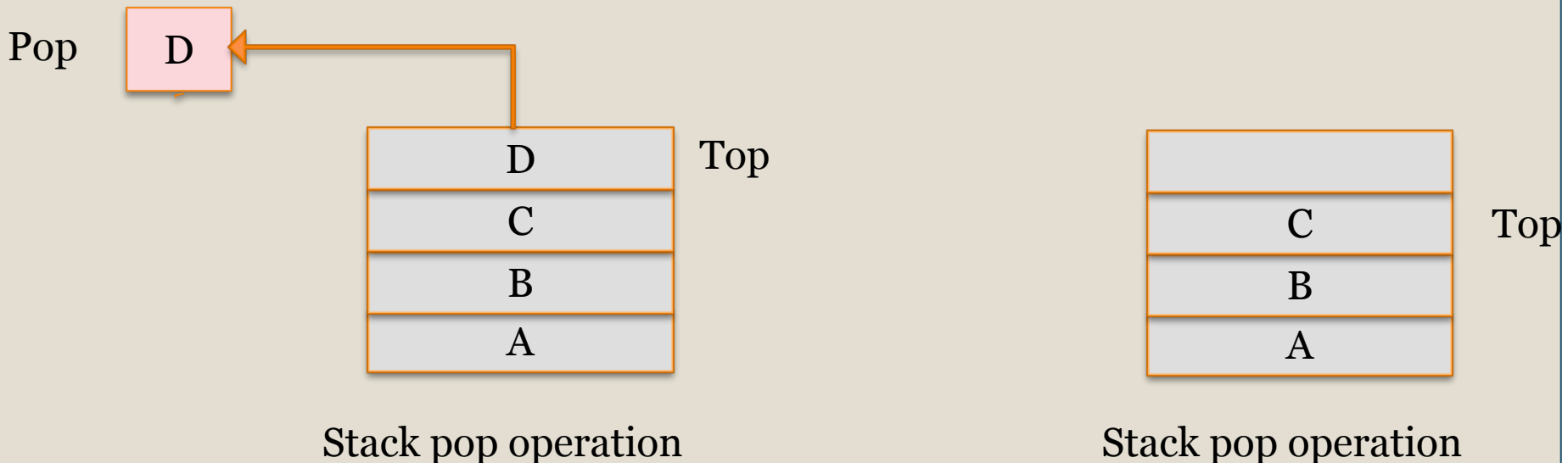
- ❑ It inserts or pushes a new data element into the stack.
- ❑ Before this operation, it is often checked if stack has enough capacity to accommodate the new item.
- ❑ If stack is full an **overflow error** would occur.
 - **Overflow:** trying to push on to an already full stack .



Pop Operation

7

- ❑ It removes an existing element from the top of stack.
- ❑ Before this operation, it must ensure that the stack is empty or not.
- ❑ If the stack contains no item (empty), popping would cause **underflow error**.
 - **Underflow:** trying to pop an empty stack.



Stack Operations and Implementations

□ Example:

- The table shows a series of stack operations and their effects on an initially empty stack S of integers.

□ Stack Implementations

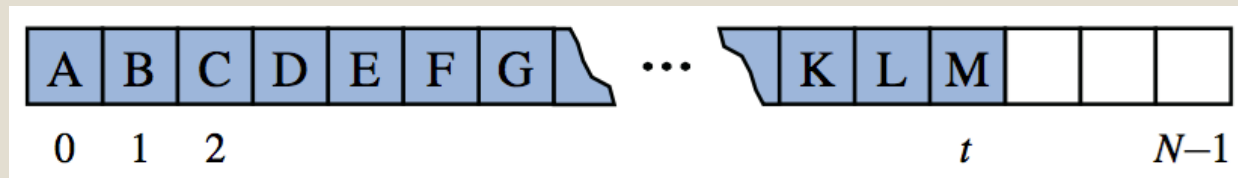
- Array based queue implementation
- Linked list-based queue implementation

Method	Return Value	Stack Contents
push(5)	—	(5)
push(3)	—	(5, 3)
size()	2	(5, 3)
pop()	3	(5)
isEmpty()	false	(5)
pop()	5	()
isEmpty()	true	()
pop()	null	()
push(7)	—	(7)
push(9)	—	(7, 9)
top()	9	(7, 9)
push(4)	—	(7, 9, 4)
size()	3	(7, 9, 4)
pop()	4	(7, 9)
push(6)	—	(7, 9, 6)
push(8)	—	(7, 9, 6, 8)
pop()	8	(7, 9, 6)

Array-Based Stack Implementation

9

- As our first implementation of the stack, we store elements in an array, named `data`, with capacity N for some fixed N .



- The bottom element of the stack is always stored in cell `data[0]`, and the top element of the stack in cell `data[t]`, where t is equal to one less than the current size of the stack.
- Recalling that arrays start at index 0 in Java, when the stack holds elements from `data[0]` to `data[t]` inclusive, it has size $t + 1$.
- By convention, when the stack is empty it will have t equal to -1
- The array implementation of a stack is simple and efficient.

Implementation

10

```
public class ArrayStack<E> {  
  
    public static final int CAPACITY=1000; // default array capacity  
    private E data [ ]; // generic array used for storage  
  
    private int t =-1; // index of the top element in stack  
  
    public ArrayStack() { this(CAPACITY); } // constructs stack with default capacity  
    public ArrayStack(int capacity) { //constructs stack with given capacity  
  
        data = (E[ ]) new Object[capacity]; //safe cast; compiler may give warning  
    }  
}
```

Implementation



```
public int size()
{
    return (t + 1); // Return Size
}
public boolean isEmpty()
{
    return (t == -1);
}
public void push(E e) throws IllegalStateException
{
    if (size() == data.length) throw new IllegalStateException("Stack is full");
    data[++t] = e; //increment t before storing new item
}
```

Implementation

12

```
public E top() {  
    if (isEmpty())  
        return null;  
    return data[t];  
}  
  
public E pop() {  
    if (isEmpty())  
        return null;  
    E answer = data[t];  
    //dereference to help garbage  
    collection  
    data[t] = null;  
  
    t--;  
    return answer;  
}
```

```
public void display(){  
    for(int i=0;i<=t;i++){  
        System.out.print(data[i]+ "  
    ");  
    }  
    System.out.println();  
}
```

Implementation



```
public static void main(String args []) {  
    arrayStacktest stack = new arrayStacktest(5);  
    stack.push(12);  
    stack.push(20);  
    stack.push(34);  
    stack.push(44);  
    stack.push(88);  
    stack.display();  
    System.out.println(stack.size());  
    System.out.println(stack.top());  
    stack.pop();  
    stack.pop();  
    stack.pop();  
    stack.pop();  
    stack.pop();  
    stack.display();  
    System.out.println(stack.size());  
    System.out.println(stack.top());  
}
```

Output :

```
12  20  34  44  88  
5  
88  
  
0  
null
```

Analyzing the Array-Based Stack Implementation

14

- ❑ Each method of the array-based stack executes a constant number of statements.
- ❑ Each method runs in constant time, that is, they each run in $O(1)$ time.
- ❑ The bellow Table shows the running times for methods of this array-based stack implementation.

Method	Running Time
size	$O(1)$
isEmpty	$O(1)$
top	$O(1)$
push	$O(1)$
pop	$O(1)$

A Drawback of the Array-Based Stack Implementation

15

❑ Limited Capacity:

- The array-based stack has a maximum capacity determined by the size of the array.

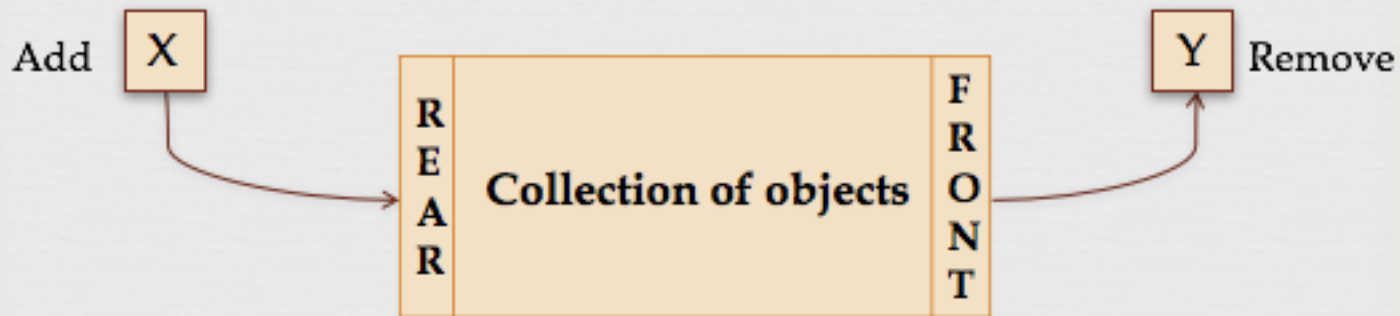
❑ Memory Usage:

- The array may allocate more memory than needed if the maximum capacity is set to accommodate rare exceptional cases with a large number of elements
- ❑ In cases where a user has a good estimate on the number of items needing to go in the stack, the array-based implementation is hard to beat. Otherwise there can be grave consequences.

Queue

16

- ❑ A queue is a collection of objects that are inserted and removed according to the *first-in, first-out (FIFO)* principle.
- ❑ Queue is a data structure with the restriction that items can be inserted only at one end called **rear** and removed at the other end called **front**.
- ❑ **FIFO** means the first item *added* to the queue will be the first one to be *removed*.
- ❑ The fundamental operations involve the “*enqueue*” and “*dequeue*”.

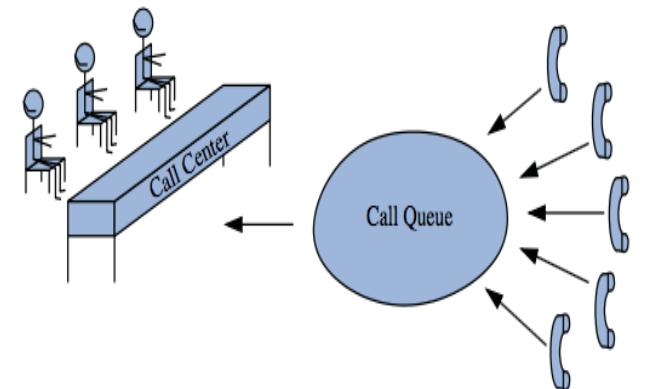
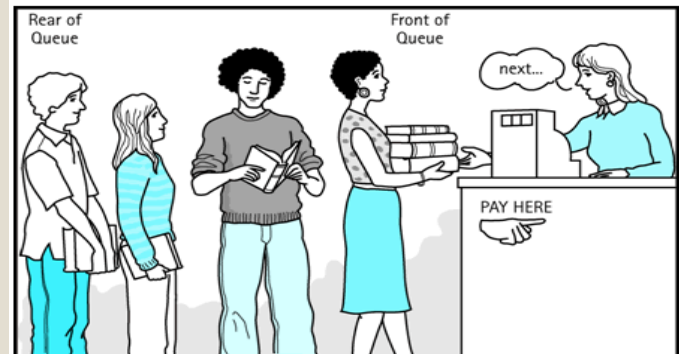
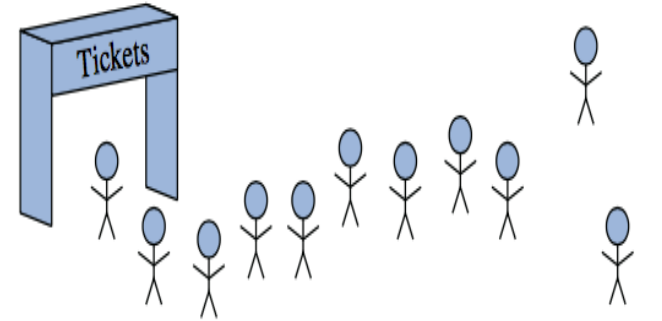


Queues Applications



Real-world examples of a first-in, first-out queue in our daily life:

- People waiting in a line at any service point such as to get an amusement park ride, and to get a ticket.
- They are used to buffer characters entered through keyboard
- CPU uses queues to schedule printing jobs
- Operating systems use queues to implement scheduling policies
- A queue would therefore be a logical choice for a data structure to handle calls to a customer service center, or a wait-list at a restaurant .
- Queues are also used by many computing devices, such as a Web server responding to requests.



Operations on Queue

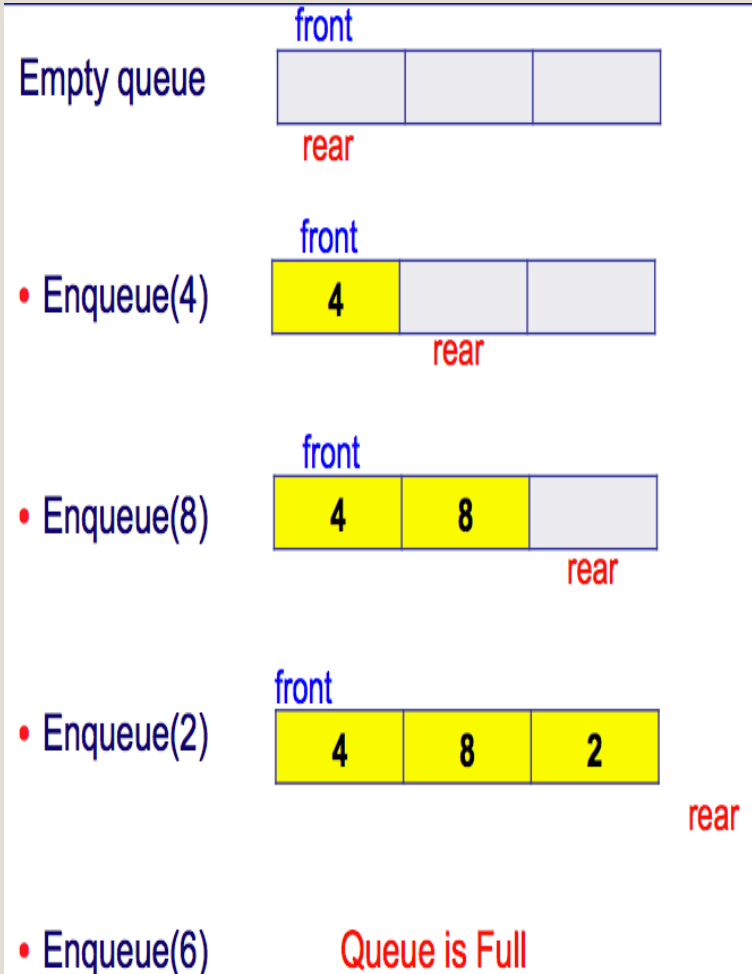
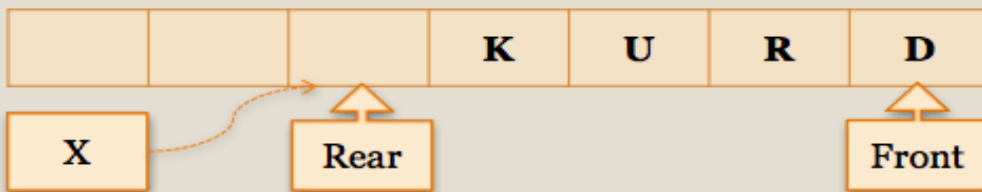


- ❑ Formally, the queue abstract data type defines a collection that keeps objects in a sequence. The *queue* abstract data type (ADT) supports the following two update methods:
 - **enqueue(*e*)**: Adds element *e* to the back of queue.
 - **dequeue()**: Removes and returns the first element from the queue (or null if the queue is empty).
- ❑ The queue ADT also includes the following accessor methods
 - **first()**: Returns the first element of the queue, without removing it (or null if the queue is empty).
 - **size()**: Returns the number of elements in the queue.
 - **isEmpty()**: Returns a Boolean indicating whether the queue is empty.

Enqueue Operation

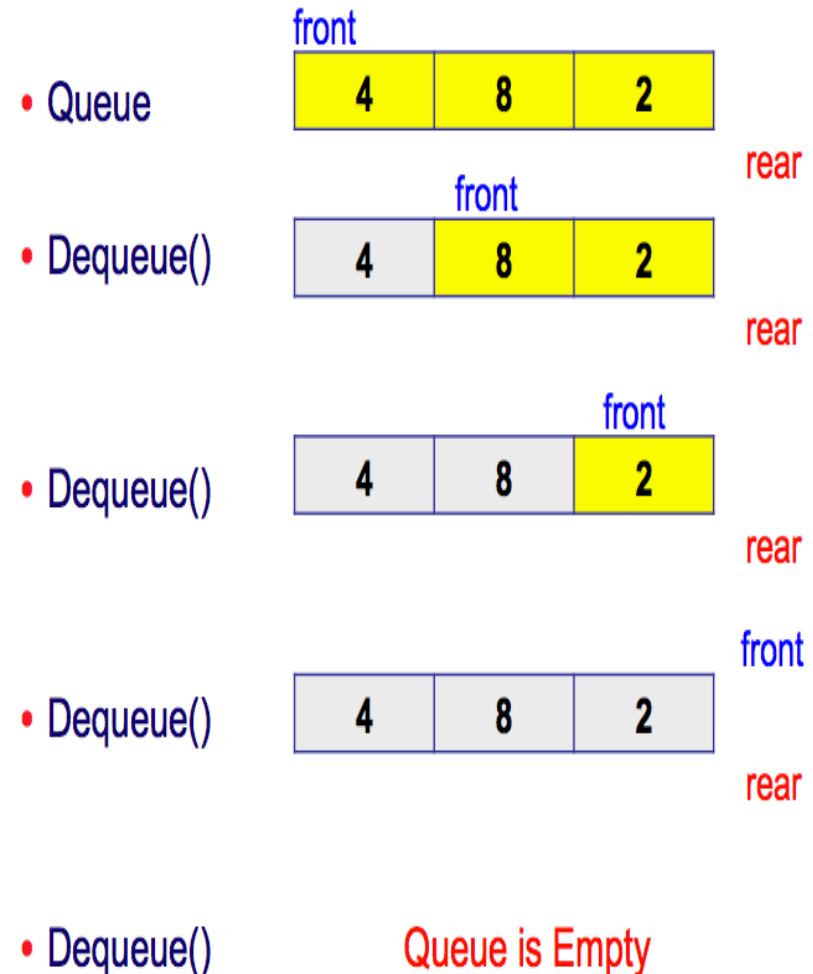
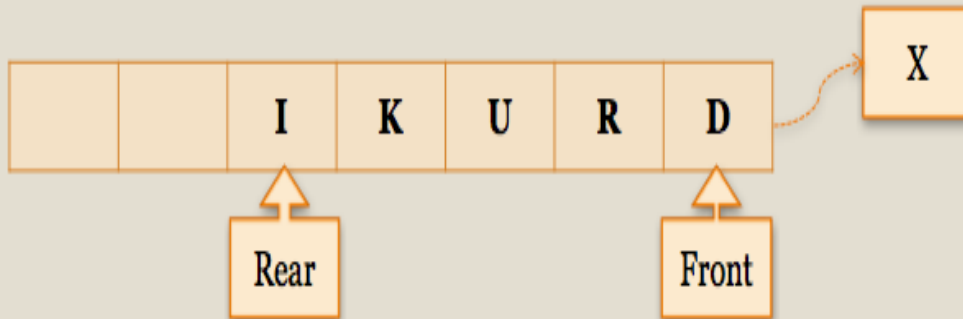


- ❑ Placing an item in a queue is called “insertion or enqueue”, which is done at the end of the queue “**rear**”.
- ❑ Before this operation, it is often checked if queue has enough capacity to accommodate the new item.
- ❑ If queue is full, add operation may result an **overflow error** .



Deque Operation

- ❑ It remove an item from the front of the queue
- ❑ Before this operation, it must ensured that queue is empty or not
- ❑ If queue is empty, **underflow** condition occur



Operations on queue



□ Example:

- The table shows a series of queue operations and their effects on an initially empty queue Q of integers.

Method	Return Value	first $\leftarrow Q \leftarrow$ last
enqueue(5)	—	(5)
enqueue(3)	—	(5, 3)
size()	2	(5, 3)
dequeue()	5	(3)
isEmpty()	false	(3)
dequeue()	3	()
isEmpty()	true	()
dequeue()	null	()
enqueue(7)	—	(7)
enqueue(9)	—	(7, 9)
first()	7	(7, 9)
enqueue(4)	—	(7, 9, 4)

Queue operations and implementation



□ Example:

- The table shows a series of queue operations and their effects on an initially empty queue Q of integers.

□ Queue Implementations

- Array based queue implementation
- Linked list-based queue implementation

Method	Return Value	first $\leftarrow$ Q $\leftarrow$ last
enqueue(5)	—	(5)
enqueue(3)	—	(5, 3)
size()	2	(5, 3)
dequeue()	5	(3)
isEmpty()	false	(3)
dequeue()	3	()
isEmpty()	true	()
dequeue()	null	()
enqueue(7)	—	(7)
enqueue(9)	—	(7, 9)
first()	7	(7, 9)
enqueue(4)	—	(7, 9, 4)

Array based queue implementation



- ❑ FIFO semantics of the Queue ADT will be implemented by using an array.
- ❑ Let's assume that as elements are inserted into a queue, we store them in an array such that the first element is at *index 0*, the second element at *index 1*, and so on.



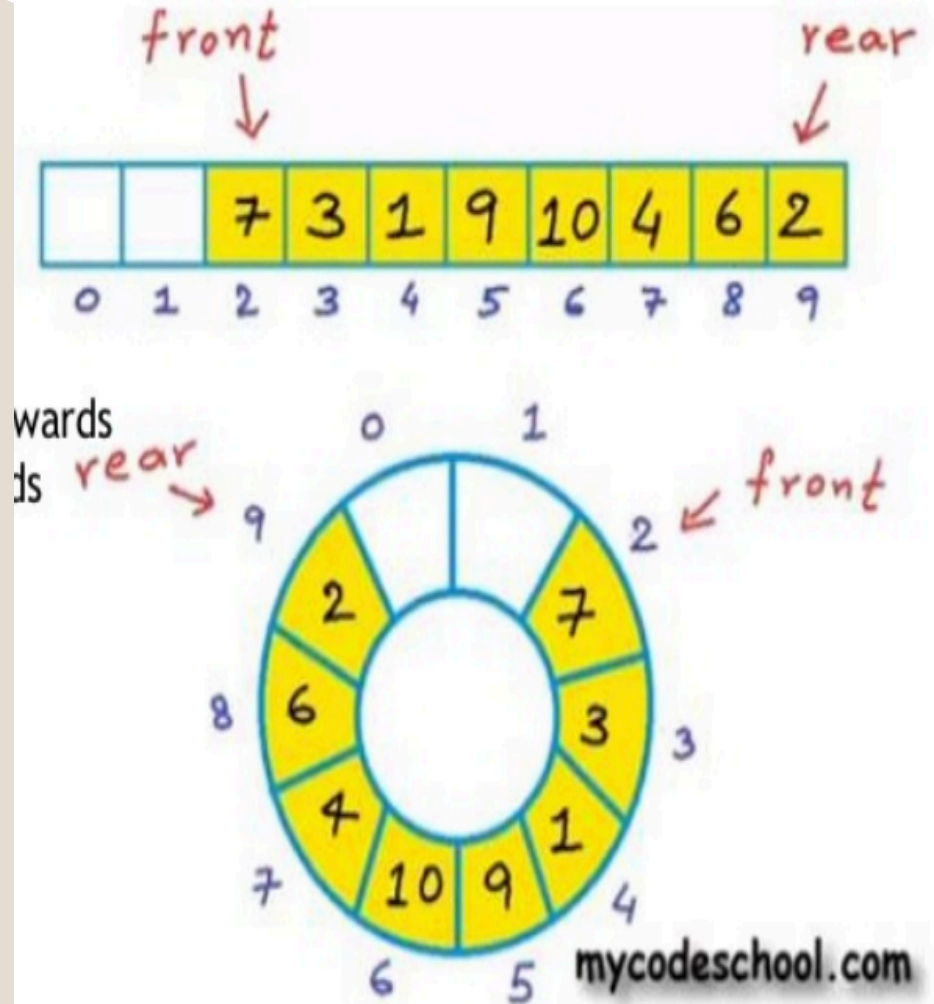
- ❑ The element to be removed is stored at *index 0* of the array.
- ❑ One strategy is to execute a *loop to shift* all other elements of the queue one cell to the *left*, so that the front of the queue is again aligned with cell 0 of the array.
- ❑ We will replace a dequeued element in the array with a *null* reference, and maintain an explicit *variable f* to represent the index of the element that is currently at the front of the queue.



- ❑ We must decide how to store additional elements in such a configuration. when there are fewer than *N* elements currently in the queue.

Using an Array Circularly Queue

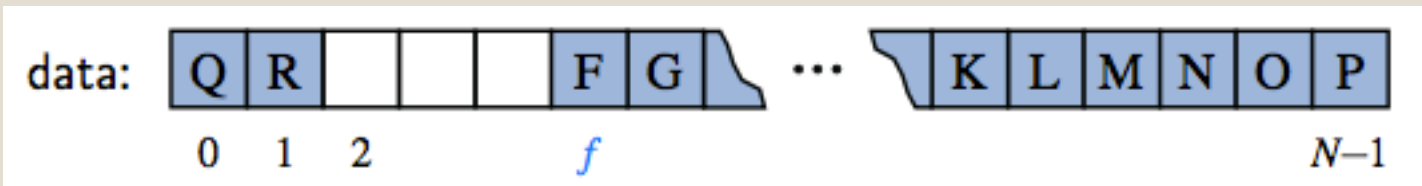
- ❑ After a sequence of additions and removals, rear of the queue will point to the last array position.
- ❑ Even though, there are empty spaces in front of the queue array, enqueue operation cannot be performed on the queue. Giving the impression that the queue is full.
- ❑ To avoid this problem wrap Front and Rear around to the beginning of the array. The result is a circular queue (sometimes called a ring buffer)



Using an Array Circularly Queue



- ❑ In developing a robust queue implementation, we allow both the front and back of the queue to drift rightward, with the contents of the queue “wrapping around” the end of an array, as necessary.
- ❑ Assuming that the array has fixed length N , new elements are enqueued toward the “end” of the current queue, progressing from the front to index $N - 1$ and continuing at index 0, then 1.



- ❑ Implementing such a circular view is relatively easy with the *modulo* operator, denoted with the symbol `%` in Java.
- ❑ When we dequeue an element and want to “advance” the front index, we use the arithmetic $f = (f + 1) \% N$.

A Java Queue Implementation



- ❑ A complete implementation of a queue ADT using an array in circular fashion is presented. Internally, the queue class maintains the following three instance variables:
 - **data:** a reference to the underlying array.
 - **f:** an integer that represents the index, within array data, of the first element of the queue (assuming the queue is not empty).
 - **sz:** an integer representing the current number of elements stored in the queue (not to be confused with the length of the array).

- ❑ We allow the user to specify the capacity of the queue as an optional parameter to the constructor.

Implement Array Queue

27

```
// instance variables
public class ArrayQueue<E> {
    private E[ ] data;      // generic array used for storage
    private int f = 0;      // index of the front element
    private int sz = 0;     // current number of elements
    public static final int CAPACITY=1000;

    // constructors
    public ArrayQueue( ) { this(CAPACITY);}      // constructs queue
    with default capacity
    public ArrayQueue(int capacity) {           // constructs queue with given
                                                capacity
    data = (E[ ]) new Object[capacity]; // safe cast; compiler may give
                                         warning
    }
```

Implement Array Queue

28

```
// methods
```

```
/** Returns the number of elements in the queue. */  
public int size() { return sz; }
```

```
/** Tests whether the queue is empty. */  
public boolean isEmpty() { return (sz == 0); }
```

```
/** Inserts an element at the rear of the queue. */  
public void enqueue(E e) throws IllegalStateException {  
    if (sz == data.length)  
        throw new IllegalStateException("Queue is full");  
    int avail = (f + sz) % data.length; // use modular arithmetic  
    data[avail] = e;  
    sz++;  
}
```

Implement Array Queue

29

```
/** Returns, but does not remove, the first element of the queue  
(null if empty). */
```

```
public E first( ) {  
    if (isEmpty())  
        return null;  
    return data[f];  
}
```

```
/** Removes and returns the first element of the queue (null if  
empty). */
```

```
public E dequeue( ) {  
    if (isEmpty())  
        return null;  
    E answer = data[f];  
    data[f] = null;  
    f = (f + 1) % data.length;  
    sz--;  
    return answer;  
}
```

```
}
```

Implement Array Queue



```
public static void main(String arg[]) {  
    QueuesArray S = new QueuesArray(5 );  
    S.enqueue(10);  
    S.enqueue(12);  
    S.enqueue(15);  
    S.enqueue(17);  
    S.enqueue(30);  
    S.disp();  
    System.out.println(S.size());  
    System.out.println(S.isEmpty());  
    S.dequeue();  
    System.out.println(S.first());  
    S.disp();  
}
```

Output :

```
10 12 15 17 30  
5  
false  
12  
12 15 17 30
```

Analyzing the Efficiency of an Array-Based Queue



- ❑ each of the queue methods in the queue array realization executes a constant number of statements involving arithmetic operations, comparisons, and assignments.
- ❑ each method in this implementation runs in $O(1)$ time.
- ❑ The bellow Table shows the running times for methods of this array-based queue implementation.

Method	Running Time
size	$O(1)$
isEmpty	$O(1)$
first	$O(1)$
enqueue	$O(1)$
dequeue	$O(1)$

Implementing a Queue with a Singly Linked List

32

```
public class LinkedQueue<E> {  
    // an empty list  
    private SinglyLinkedList<E> list = new SinglyLinkedList<E>();  
  
    public LinkedQueue() { } // new queue relies on the initially empty  
    list  
    public int size() { return list.size(); }  
    public boolean isEmpty() { return list.isEmpty(); }  
    public void enqueue(E element) {  
        list.addLast(element); }  
    public E first() { return list.first(); }  
    public E dequeue() {  
        return list.removeFirst();  
    } }  
}
```




Any Questions?

